

OpenFormulare – Plugin-Entwicklung

Stand: Mai 2026 · Plugin-API v1 · Lizenz: MIT

OpenFormulare ist als Open-Source-Plattform für digitale Formulare und Dialoge konzipiert. Ab Version 1.x ist das Tool über ein offizielles Plugin-System erweiterbar – ohne den Kern verändern zu müssen.

Dieses Dokument richtet sich an Entwickler:innen, die eigene Plugins schreiben oder bestehende Erweiterungen verstehen wollen.

1. Was ist ein OpenFormulare-Plugin?

Ein **Plugin** ist ein Node.js-Paket, das beim Start des Backends geladen wird und Funktionalität ergänzt, ohne den OpenFormulare-Kern zu modifizieren. Typische Anwendungen:

- Anbindung externer Systeme (CalDAV, CRM, Buchhaltung, ...)
- Zusätzliche Bewertungsbögen (DiNo bleibt im Kern – externe Bewertungsmodule können aber als Plugin angebunden werden)
- Eigene Import-/Export-Formate
- Webhooks, die bei Dialog-Einreichung externe Dienste benachrichtigen
- Eigene Feld-Typen für den Dialog-Editor

Plugins werden **in-process** geladen: sie laufen im selben Node-Prozess wie das Backend und haben Zugriff auf die offiziellen SDK-APIs. Der Kern setzt darauf, dass Plugin-Autoren vertrauenswürdig sind. Ein buggy Plugin reißt zwar nicht den Server mit (Hooks sind in try/catch gekapselt), kann aber bewusst Schaden anrichten – Operator:innen sollten nur geprüfte Plugins aktivieren.

Für strikt isolierte Drittanbieter-Erweiterungen ist ein zukünftiges Webhook-basiertes Plugin-System geplant.

2. Plugin-Struktur

Ein Plugin lebt in einem eigenen Unterordner unter `plugins/`:

```
plugins/
├── <plugin-id>/
│   ├── plugin.json      ← Manifest (Pflicht)
│   ├── package.json     ← npm-Paket-Definition
│   ├── tsconfig.json    ← TypeScript-Build-Konfiguration
│   └── src/
│       └── index.ts      ← Quellcode
```

```
└─ dist/
  └─ index.js      ← Kompilierter Entry-Point (von tsc erzeugt)
```

plugin.json (Manifest)

```
{
  "id": "my-plugin",
  "name": "Mein Plugin",
  "version": "1.0.0",
  "description": "Was das Plugin tut.",
  "author": "Mein Name",
  "homepage": "https://example.com/my-plugin",
  "main": "dist/index.js",
  "settings": [
    {
      "key": "apiToken",
      "label": "API-Token",
      "type": "password",
      "required": true
    }
  ]
}
```

Feld	Pflicht	Beschreibung
id	ja	Eindeutige Kennung. Nur Kleinbuchstaben, Zahlen, Bindestriche.
name	ja	Anzeigename in der Admin-UI.
version	ja	SemVer.
description	nein	Kurztext für die Admin-UI.
author	nein	Autor:in / Organisation.
homepage	nein	Link zum Repository / Marketing-Seite.
main	nein	Pfad zum kompilierten Entry. Default: <code>dist/index.js</code> .
settings	nein	Liste der Konfigurationsfelder, siehe Abschnitt 5.

Entry-Point (src/index.ts)

Das Plugin exportiert **default** ein Objekt der Form `PluginModule`:

```
import type { PluginModule } from '@openformulare/plugin-sdk';

const plugin: PluginModule = {
  id: 'my-plugin',
  hooks: {
    'dialog:submitted': async (event, ctx) => {
      ctx.log.info('Dialog eingereicht', { dialogId: event.dialogId });
    }
  }
};
```

```

    },
  },
  routes: (router, ctx) => {
    router.get('/status', (_req, res) => {
      res.json({ ok: true });
    });
  },
  fieldTypes: [
    {
      id: 'rating-emoji',
      label: 'Emoji-Bewertung',
      behavior: 'select',
    },
  ],
  onActivate: (ctx) => {
    ctx.log.info('Plugin aktiviert');
  },
  onDeactivate: (ctx) => {
    ctx.log.info('Plugin deaktiviert');
  },
};

export default plugin;

```

Wichtig: Der `@openformulare/plugin-sdk`-Import ist als `import type` nur zur Compile-Zeit nötig. Im kompilierten JS bleiben keine externen `require`s zurück. Das macht Plugins komplett selbstständig und unabhängig von der OpenFormulare-Code-Basis zur Laufzeit.

3. Plugin registrieren und verwalten

Installation

1. Plugin-Verzeichnis nach `plugins/<plugin-id>/` kopieren oder per `git clone` einbinden.
2. Plugin bauen: `cd plugins/<plugin-id> && npm install && npm run build`
3. Backend neu starten. Beim Start scannt OpenFormulare das `plugins/`-Verzeichnis und legt für jedes gefundene Manifest einen Eintrag in der `plugins`-Tabelle an (Status: `enabled = false`).

Aktivieren / Deaktivieren

In der Admin-UI unter `/admin/plugins`:

- Liste aller installierten Plugins
- Toggle-Button "Aktivieren" / "Deaktivieren"
- Settings-Formular pro Plugin (auto-generiert aus `manifest.settings`)
- Fehleranzeige, falls ein Plugin beim Laden Probleme hatte

Aktivierung läuft synchron über die Admin-API:

```
POST /api/admin/plugins/<plugin-id>/enable      (Authorization: Bearer ...)  
POST /api/admin/plugins/<plugin-id>/disable
```

Beim Aktivieren wird `onActivate(ctx)` ausgeführt, beim Deaktivieren `onDeactivate(ctx)`. Beide Hooks sind optional.

Dynamisch geladen oder muss man das Backend neustarten?

- **Aktivierung/Deaktivierung:** zur Laufzeit, kein Neustart nötig
- **Neue Plugin-Installation oder Code-Änderungen:** Backend-Neustart
- **Settings-Änderungen:** zur Laufzeit (Settings werden bei jedem Hook-Call frisch aus der DB gelesen)

4. Hooks / Events

OpenFormulare feuert bei zentralen Lifecycle-Punkten Events. Ein Plugin kann auf jedes Event reagieren, indem es eine Funktion unter dem entsprechenden Schlüssel in `hooks` definiert.

Event	Wann	Payload (Auszug)
<code>dialog:beforeSubmit</code>	Direkt vor dem Speichern/Versenden einer Submission	<code>dialogId</code> , <code>dialog</code> , <code>submission</code>
<code>dialog:submitted</code>	Nach erfolgreichem Versand (DiNo / E-Mail)	<code>dialogId</code> , <code>dialog</code> , <code>submission</code> , <code>submittedAt</code>
<code>rating:created</code>	Wenn ein Bewertungsbogen abgegeben wird	<code>ratingId</code> , <code>templateId</code> , <code>context</code>
<code>appointment:requested</code>	Wenn ein Plugin/Modul einen Termin-Wunsch meldet	<code>proposedAt</code> , <code>durationMinutes</code> , <code>participants</code>
<code>data:exported</code>	Nach erfolgreichem Export (JSON, CSV, ...)	<code>format</code> , <code>scope</code> , <code>resourceId</code> , <code>payloadBytes</code>
<code>data:imported</code>	Nach erfolgreichem Import	<code>format</code> , <code>scope</code> , <code>resourceId</code> , <code>payloadBytes</code>

Hook-Handler haben die Signatur:

```
type HookHandler<E> = (event: PluginEventMap[E], ctx: PluginContext) => void | Prom
```

Wichtige Eigenschaften

- **Asynchron parallel:** Mehrere Plugins, die auf dasselbe Event reagieren, laufen gleichzeitig (`Promise.all`).

- **Fehlertolerant:** Wirft ein Plugin in einem Hook eine Exception, wird diese geloggt und im Admin-UI angezeigt – andere Plugins laufen weiter.
- **Mutation:** Bei `dialog:beforeSubmit` darf das Plugin die `submission` direkt mutieren. Mutationen propagieren in den Persistenzpfad.
- **Reihenfolge:** Plugins werden in alphabetischer Reihenfolge ihrer ID aufgerufen. Verlasse dich nicht darauf – designe Plugins unabhängig.

5. Plugin-Einstellungen

Jedes Plugin kann eigene Konfigurationswerte definieren. Diese werden in der `plugin_settings`-Tabelle (Schlüssel `plugin_id` + `key`) als String gespeichert.

Schema-Definition (`manifest.settings`)

```
[
  {
    "key": "calendarUrl",
    "label": "Kalender-URL",
    "type": "url",
    "required": true,
    "description": "CalDAV-Endpoint, z. B. https://nextcloud.example.com/remote.php",
  },
  {
    "key": "durationMinutes",
    "label": "Termin-Dauer",
    "type": "number",
    "default": 30,
    "min": 5,
    "max": 480
  },
  {
    "key": "active",
    "label": "Aktiv",
    "type": "boolean",
    "default": true
  }
]
```

Unterstützte `type`-Werte: `string`, `number`, `boolean`, `password`, `url`, `select`. Bei `select` muss zusätzlich `options: [{ value, label }]` gesetzt sein.

Lesen und Schreiben

Im Plugin-Code:

```
const apiToken = ctx.settings.get<string>('apiToken');
const all = ctx.settings.getAll(); // { apiToken: '...', durationMinutes: '30', ...
ctx.settings.set('lastSync', new Date().toISOString());
```

Werte werden immer als String gespeichert. Plugin-Autoren sind dafür verantwortlich, sie ggf. in Zahl/Bool zu casten.

UI

Das Admin-UI unter `/admin/plugins` rendert pro Plugin automatisch ein Formular auf Basis der Schema-Definition – keine Custom-React-Komponente nötig.

6. Eigene Routen

Plugins können zusätzliche HTTP-Endpoints definieren:

```
const plugin: PluginModule = {
  id: 'my-plugin',
  routes: (router, ctx) => {
    router.get('/status', (_req, res) => res.json({ ok: true }));
    router.post('/sync', async (req, res) => {
      // Plugin-spezifische Logik
      res.json({ synced: true });
    });
  },
};
```

Diese Routes werden unter `/api/plugins/<plugin-id>/...` gemountet (nur wenn das Plugin aktiv ist). Die Authentifizierung übernimmt das Plugin selbst – z. B. mit einem geteilten Geheimnis aus den Settings.

7. Eigene Feld-Typen

Plugins können neue Feld-Typen für den Dialog-Editor registrieren:

```
fieldTypes: [
  {
    id: 'iban',
    label: 'IBAN',
    description: 'Bankverbindung mit Validierung',
    behavior: 'text',
    defaultProps: { placeholder: 'DE...' },
  },
],
```

Im Frontend werden Plugin-Feld-Typen über die generische `PluginField`-Komponente gerendert. Das `behavior`-Feld bestimmt das HTML-Eingabe-Element (`text`, `number`, `textarea`, `select`, `checkbox`, `date`).

Für Feld-Typen mit komplett eigener React-UI ist in einer späteren Plugin-API-Version "Frontend Extensions via Module Federation" geplant. v1 deckt einfache, validierte Eingabe-Komponenten ab.

8. Beispielpugin: WebDAV-Kalender

Das Plugin `webdav-calendar` ist Erstanbieter und im Repository mitgeliefert. Ab Version **1.1** demonstriert es zusätzlich, wie ein Plugin einen eigenen Feld-Typ mit komplexer UI ergänzt – inklusive Slot-Berechnung und Live-Abgleich gegen den CalDAV-Server.

Was das Plugin kann

- Bindet einen **oder mehrere** CalDAV-Kalender an (Standard-Kalender via Plugin-Settings, weitere als JSON-Liste).
- Definiert konfigurierbare **Buchungs-Zeitfenster** (Wochentage, Uhrzeiten, optional pro Kalender, optional auf Datumsbereiche beschränkt).
- Stellt den neuen Feld-Typ **Kalender / Termin** für den Dialog-Editor bereit. Im Dialog zeigt das Feld einen Tag-/Zeit-Picker im Stil der Nextcloud-Terminfindung.
- Liest beim Picker-Aufruf via **CalDAV REPORT** alle bestehenden Termine im Buchungs-Horizont und zieht sie als belegte Zeiten ab – Doppelbuchungen werden ausgeschlossen.
- Beim Absenden des Dialogs legt der `dialog:submitted`-Hook für jeden gewählten Slot automatisch einen Termin im richtigen Kalender an.
- **Legacy-Pfad**: Dialoge ohne Kalender-Feld nutzen weiterhin die beiden alten Felder `termin_datum` und `termin_uhrzeit`.

HTTP-Endpoints des Plugins

Endpoint	Auth	Funktion
GET <code>/api/plugins/webdav-calendar/calendars</code>	öffentlich	Liste verfügbarer Kalender (<code>id</code> , <code>label</code>).
GET <code>/api/plugins/webdav-calendar/slots?calendarId=...&from=...&to=...</code>	öffentlich	Freie Slots im Zeitraum, vom CalDAV-Server abgeglichen.
POST <code>/api/plugins/webdav-calendar/test</code>	öffentlich	Smoketest: legt morgen einen Test-Termin an.

Plugin-Settings

Setting	Typ	Beschreibung
calendarUrl	URL	Standard-Kalender (Pflicht).
username	String	CalDAV-User für den Standard-Kalender.
password	Password	App-Token / Passwort.
calendars	JSON-Str	Optional: weitere Kalender als Array [<code>{id,label,url,username,password}</code>].
timezone	String	IANA-Zeitzone für Buchungsfenster (Default <code>Europe/Berlin</code>).
bookingWindows	JSON-Str	Liste von Buchungsfenstern. Schema siehe unten.
bookingHorizonDays	Zahl	Wie weit in die Zukunft Buchungen erlaubt sind (Default 60).
minLeadTimeMinutes	Zahl	Mindest-Vorlauf eines Slots ab "jetzt" (Default 60).
dateFieldId / timeFieldId	String	Legacy: Feld-IDs für Dialoge ohne Kalender-Feld.
summaryTemplate	String	Termin-Titel-Vorlage. Platzhalter: <code>{dialogTitle}</code> , <code>{dialogId}</code> , <code>{participantName}</code> .

bookingWindows -Schema

```
[
  { "weekdays": [1, 2, 3, 4, 5], "from": "09:00", "to": "12:00", "slotMinutes": 30
  { "weekdays": [1, 2, 3, 4, 5], "from": "14:00", "to": "17:00", "slotMinutes": 30
  { "weekdays": [6], "from": "10:00", "to": "14:00", "slotMinutes": 60, "calendarId
  {
    "weekdays": [3], "from": "08:00", "to": "10:00", "slotMinutes": 15,
    "dateFrom": "2026-06-01", "dateTo": "2026-06-30"
  }
}
```

- `weekdays` : 0 = Sonntag, 1 = Montag, ..., 6 = Samstag
- `from` / `to` : Wanduhrzeit in der konfigurierten `timezone`
- `slotMinutes` : Granularität des Pickers (auch die Termin-Dauer beim Anlegen des Events)
- `calendarId` (optional): Fenster gilt nur für den Kalender mit dieser ID
- `dateFrom` / `dateTo` (optional): Fenster gilt nur in diesem Datumsbereich (z. B. Sondersprechzeiten)

Den Kalender-Picker in einem Dialog nutzen

1. Plugin in der Admin-UI aktivieren und Settings ausfüllen.
2. Im Dialog-Editor neues Feld vom Typ **Kalender / Termin** anlegen.
3. (Optional) Im Feld-JSON `calendarId` setzen, um einen anderen als den Standard-Kalender zu verwenden.
4. Veröffentlichen – der Endkunde sieht den Picker und wählt einen freien Slot.

5. Nach dem Absenden erstellt das Plugin im Hintergrund einen CalDAV-Termin.

Test ohne Submission

```
# Liste der Kalender
curl http://localhost:3001/api/plugins/webdav-calendar/calendars

# Freie Slots der nächsten 14 Tage
curl 'http://localhost:3001/api/plugins/webdav-calendar/slots?calendarId=default'

# Test-Termin
curl -X POST http://localhost:3001/api/plugins/webdav-calendar/test
```

Erweitern auf andere Kalender-Anbieter

Der CalDAV-Layer ist im Plugin gekapselt (`fetchBusy()` , `uploadEvent()`). Für andere Anbieter (Google Calendar API, Microsoft Graph, eigene HTTP-Schnittstellen) lassen sich diese beiden Funktionen in einem neuen Plugin oder einem Fork dieses Plugins ersetzen, ohne dass die Slot-Berechnungs- und Booking-Window-Logik angepasst werden muss.

9. Eigenes Plugin entwickeln – Schritt für Schritt

a) Projekt anlegen

```
mkdir -p plugins/my-plugin/src
cd plugins/my-plugin
```

b) `package.json`

```
{
  "name": "openformulare-plugin-my-plugin",
  "version": "1.0.0",
  "private": true,
  "main": "dist/index.js",
  "scripts": {
    "build": "tsc -p tsconfig.json"
  },
  "devDependencies": {
    "typescript": "^5.4.0",
    "@types/node": "^20.0.0"
  }
}
```

c) `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "CommonJS",
    "moduleResolution": "Node",
    "outDir": "dist",
    "rootDir": "src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "paths": {
      "@openformulare/plugin-sdk": ["../../backend/src/plugins"]
    },
    "baseUrl": "."
  },
  "include": ["src/**/*"]
}
```

d) plugin.json

Siehe Abschnitt 2.

e) src/index.ts

```
import type { PluginModule } from '@openformulare/plugin-sdk';

const plugin: PluginModule = {
  id: 'my-plugin',
  hooks: {
    'dialog:submitted': async (event, ctx) => {
      ctx.log.info('Hello from my-plugin', { dialogId: event.dialogId });
    },
  },
};

export default plugin;
```

f) Bauen & Testen

```
npm install
npm run build
# Backend neu starten - Plugin erscheint unter /admin/plugins
```

g) Plugin paketieren

Für andere Operator:innen:

```
tar -czf my-plugin-1.0.0.tgz plugins/my-plugin
```

Empfehlung: Plugin als eigenes Git-Repo veröffentlichen, sodass andere Setups es per Submodule oder einfacher `git clone` einbinden können.

10. Sicherheitshinweise

- Plugins laufen mit voller Backend-Berechtigung. Nur Plugins aktivieren, deren Code geprüft wurde.
 - Settings vom Typ `password` werden in Klartext in der DB gespeichert (verschlüsselte Spalten sind eine geplante Erweiterung). Die DB selbst ist standardmäßig durch das `SQLITE_PATH`-Volume und das OS-Berechtigungsmodell geschützt.
 - Hook-Handler sollten **idempotent** sein – z. B. werden bei einem erneuten Start abhängige externe Aufrufe (Webhooks, CalDAV-Inserts) unter Umständen wiederholt.
 - Plugin-Routen unter `/api/plugins/<id>/...` haben **keine** vom Kern erzwungene Authentifizierung. Plugins, die sensible Endpoints bereitstellen, müssen das selbst tun.
-

11. Plugin-API-Versionierung

Die hier beschriebene API ist **v1**. Breaking Changes werden:

- in einer neuen Major-Version der OpenFormulare-Plugin-API gebündelt
- mindestens 6 Monate vor Release angekündigt
- mit Migration-Guide im `docs/`-Verzeichnis veröffentlicht

Die SDK-Typen sind im Repository unter `backend/src/plugins/types.ts` dokumentiert.

12. Hilfe und Beiträge

- **Issues / Feature-Requests:** GitHub Issues im OpenFormulare-Repo
- **Plugin-Beispiele:** `plugins/`-Verzeichnis, beginnend mit `webdav-calendar`
- **Pull Requests** für die Plugin-API: bitte das Label `plugin-api` verwenden, damit das Core-Team frühzeitig drüberschaut.

Viel Erfolg beim Erweitern von OpenFormulare!